TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE 4303 — Principios de Sistemas Operativos

Proyecto 1:
Calendarizador de Tiempo Real Heterogéneo

Fecha de asignación:	22 de setiembre 2026	Fecha de entrega:	22 de octubre 2026
Grupo:	4 personas	Profesor:	Luis Barboza Artavia

1. Atributo a evaluar

Trabajo en Equipo: Funciona de manera efectiva como individuo y como miembro o líder en equipos diversos e inclusivos y en entornos multidisciplinarios, cara a cara, remotos y distribuidos.

2. Objetivo

- Diseñar e implementar un sistema de calendarización de tiempo real en el que una computadora reparte un conjunto de tareas entre microcontroladores de distinta capacidad (ESP32, Raspberry Pi Pico o similar - no Arduino) y cada nodo ejecuta sus tareas con un calendarizador local. El sistema deberá implementar RMS y EDF. El algoritmo se selecciona mediante la configuración al desplegar las tareas.

3. Descripción

El sistema, como se muestra en la Figura 1, se compone de dos tipos de elementos:

1. El host (Computadora / Linux): recibe la carga de trabajo, calibra los nodos, decide en qué nodo se ejecuta cada tarea, comprueba que sea calendarizable, vigila el estado de los nodos y genera el reporte.

2. Los nodos (ESP32, Raspberry Pi Pico o similar - no Arduino): ejecutan las tareas que se les asignan mediante un calendarizador local de tiempo real desarrollado por el grupo. El nodo recibe las tareas y decide qué se ejecuta en cada instante. El host no participa en la calendarización en tiempo de ejecución: no decide qué tarea corre en cada tick.

Cada grupo debe utilizar al menos dos nodos distintos: un ESP32, una Raspberry Pi Pico o similar - no Arduino, conectados de forma serial al host.

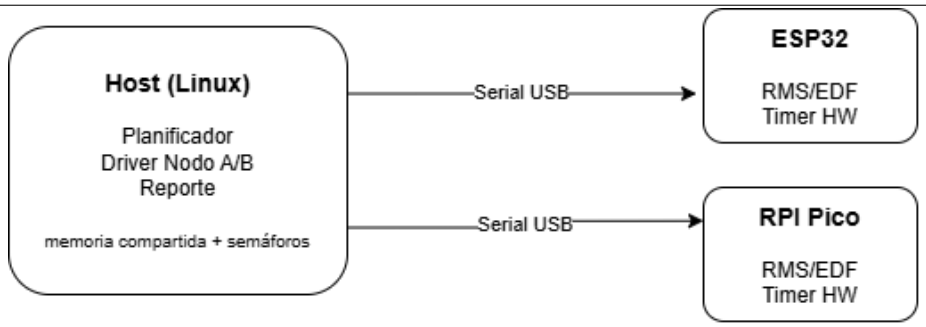


Figura 1: Arquitectura del sistema.

3.1. Modelo de tareas

- **Tarea periódica:** se define por un identificador, un trabajo W (en unidades de trabajo), un período T, un plazo relativo D y una criticidad (es un número entero. Mayor valor implica mayor criticidad). Libera un trabajo cada T ticks.
- **Tarea esporádica:** igual que la periódica, pero T representa el intervalo mínimo entre llegadas y la liberación es provocada por un evento externo (Sección 3.7).
- **Unidad de trabajo (UT):** medida abstracta de cómputo. Se les dará un archivo `ut.c` y `ut.h` que contienen la función `unidad_trabajo()`. Estos archivos deberán incorporarse sin modificaciones a cada nodo.
- **Tick:** intervalo fijo de calendarización generado por un temporizador de hardware. Su duración se define en el archivo de configuración (10 ms por defecto).
- **Costo por nodo:** durante la calibración cada nodo determina cuántas UT puede ejecutar por tick (s_n). El costo en ticks de la tarea i en el nodo n es

$$C(i,n) = \left\lceil \frac{W(i)}{s_n} \right\rceil \tag{1}$$

  Un nodo más lento requiere más ticks para completar el mismo trabajo.

Las tareas se describen mediante un archivo de texto, una tarea por línea, con campos separados por comas:

```
id,tipo,W,T,D,criticidad
```

donde `tipo` es `P` (periódica) o `S` (esporádica).
Ejemplo:

```
3,P,40,20,20,2
```

Este formato es fijo, ya que será utilizado para cargar conjuntos de tareas durante la revisión.

3.2. Nodos - Calendarizador local de tiempo real

- El calendarizador local debe ser desarrollado por el grupo. En cada tick (interrupción del temporizador de hardware) decide qué tarea avanza y ejecuta un tramo de trabajo equivalente a un tick.
- Las tareas deben implementarse de manera que su trabajo pueda avanzar un tick a la vez (por ejemplo, mediante máquinas de estados), obteniendo apropiatividad a nivel de tick. La apropiación solo ocurre al finalizar un tick. No se requiere interrupción de una tarea a mitad del trabajo correspondiente a un tick.
- El algoritmo de calendarización (RMS y EDF) se selecciona mediante la configuración al desplegar las tareas. El grupo debe definir y documentar el criterio de desempate (debe ser consistente durante toda la ejecución).
- En RMS, las tareas con menor período tienen mayor prioridad, y dichas prioridades permanecen fijas durante toda la ejecución.
- Cada nodo debe soportar al menos ocho tareas simultáneas (periódicas y esporádicas).
- Si un trabajo no finaliza antes de su plazo absoluto, el calendarizador debe registrar el evento, activar el indicador de plazo perdido y abortar ese trabajo, continuando con los demás.
- Los nodos deben seguir ejecutando sus tareas aun cuando el host se detenga o desconecte.
- No se permite utilizar `delay()`, `sleep` ni *busy waiting* como mecanismo de temporización, ni delegar en un RTOS la decisión del orden de ejecución (prioridades, colas o retardos).

3.3. Calibración

- Al iniciar el sistema, el host solicita la calibración de cada nodo.
- Cada nodo mide cuántas UT puede ejecutar por tick, dejando margen para el calendarizador y el envío de eventos, e informa ese valor al host.
- Con estas mediciones el host calcula el costo de cada tarea para cada nodo.
- Las mediciones deben repetirse en cada ejecución y no pueden estar fijadas en el código.

3.4. Host - Planificación y admisión

- **Asignación (particionado):** cada tarea se asigna a un único nodo. Deben implementarse al menos dos heurísticas. Las tareas se ordenan de mayor a menor utilización medida en el nodo de mayor s_n (desempate por criterio del grupo).
  - First-Fit: se recorren los nodos en orden decreciente de s_n y se elige el primero donde la tarea pasa la admisión.
  - Worst-Fit: entre los nodos donde pasa la admisión, se elige el que queda con mayor capacidad libre después de asignarla.
- **Prueba de admisión por nodo:**
  - EDF: la suma de C/T de las tareas del nodo debe ser menor o igual a 1.
  - RMS: la suma de C/T sea menor o igual a $n(2^{(1/n)} - 1)$. Esta prueba se utilizará como criterio de admisión del proyecto.
- Si una tarea no cabe en ningún nodo, debe rechazarse e informarse el motivo.

3.5. Modos de operación

- **Modo automático:** la carga inicial se obtiene del archivo de tareas.
- **Modo manual:** el usuario puede agregar tareas durante la ejecución. El host realiza nuevamente la admisión y, si procede, envía la nueva tarea al nodo correspondiente sin detener la ejecución del sistema.

3.6. Comunicación y sincronización

Host y nodos:

- La comunicación utiliza un protocolo serial (USB).
- Cada trama debe contener: marca de inicio, tipo, longitud de la carga útil, carga útil (contenido del mensaje) y CRC (verificación de redundancia cíclica, por ejemplo CRC-16).
- Los mensajes mínimos son: solicitud y respuesta de calibración, despliegue de tareas, inicio, evento, reporte de tick, disparo de tarea esporádica, detención y acuse de recibo.
- El formato exacto debe documentarse.
- Las tramas corruptas se descartan y se reportan.

- Los eventos incluyen marca de tiempo en ticks desde el mensaje de inicio.
- El envío de eventos no debe provocar pérdida de plazos. Se recomienda utilizar un búfer circular en RAM y transmitir en segundo plano.
- La duración del tick se envía a cada nodo antes de la calibración (por ejemplo, en la solicitud de calibración).

Procesos del host:

- El host debe estar compuesto por al menos cuatro procesos: planificador, un controlador por nodo y proceso de reporte.
- La comunicación entre ellos utiliza memoria compartida POSIX (`shm_open`, `mmap`) y semáforos POSIX.
- La cola de eventos entre drivers (productores) y reporte (consumidor) debe ser acotada y sincronizada.
- No se permite busy waiting.
- Al finalizar, deben liberarse todos los recursos y cerrarse los puertos seriales.

Sincronización temporal:

- El mensaje de inicio establece el tick cero en todos los nodos.
- Se tolera un pequeño desfase entre nodos, el cual debe medirse y documentarse.

3.7. Tareas esporádicas e interrupciones externas

- Cada nodo dispone de un botón conectado a un pin con interrupción (tome en cuenta el *debounce*). Cada nodo puede tener como máximo una tarea esporádica, liberada por el botón de ese nodo.
- Al presionarlo se libera un trabajo de la tarea esporádica correspondiente.
- La rutina de interrupción debe limitarse a registrar la liberación. El calendarizador realiza el resto del procesamiento.
- Si un evento ocurre antes del intervalo mínimo permitido, debe ignorarse y contabilizarse.
- Para la admisión, la tarea esporádica se considera periódica con período igual a dicho intervalo mínimo.
- El reporte debe incluir la latencia de respuesta de estas tareas.

3.8. Desconexión de nodos

- Cada nodo envía un reporte de tick cada N ticks, donde N es configurable.
- Si el host deja de recibir un número configurable de reportes consecutivos, declara el nodo caído.
- Ante esta situación, el host replanifica las tareas utilizando la misma heurística y el mismo criterio de admisión. Importante: las tareas del nodo sobreviviente se mantienen, las del nodo caído son las que se intenta reubicar.
- Si no todas las tareas caben en el nodo restante, deben rechazarse primero las de menor criticidad. El estado de ejecución de las tareas del nodo caído no necesita recuperarse. Las tareas reubicadas reinician su ejecución en el nuevo nodo.
- Para probar la falla, se desconectará el cable o mediante un comando del host.
- Ninguna entrada inválida debe provocar errores de acceso a memoria ni terminaciones abruptas.
- Las tareas esporádicas no se reubican tras la caída del nodo y se reportan como rechazadas.

3.9. Hardware

Cada nodo debe reflejar físicamente lo que ocurre: la tarea en ejecución (por ejemplo, codificada en binario en LEDs), un LED de plazo perdido y el botón de la tarea esporádica. Se permite agregar una pantalla u otros elementos, pero no es necesario.

3.10. Reporte

El usuario puede finalizar la ejecución en cualquier momento presionando la tecla `q`.

La terminación debe ser elegante: detener nodos, finalizar procesos, liberar recursos y generar el reporte.

El reporte debe incluir:

1. Diagrama de Gantt por nodo.
2. Liberaciones, ejecución, plazos y fallas.
3. Estadísticas por tarea.
4. Utilización teórica y medida por nodo.
5. Cambios de tarea.

6. Tareas aceptadas y rechazadas con su motivo.

El reporte puede implementarse en cualquier formato, siempre que reciba los datos mediante la cola de eventos en memoria compartida.

3.11. Configuración

Un archivo de configuración debe indicar: algoritmo (RMS o EDF), heurística de asignación, duración del tick, puertos seriales de cada nodo, periodo del reporte de tick (en ticks), número de reportes consecutivos perdidos para declarar una falla y ruta del archivo de tareas. La interacción con el usuario debe ser lo más sencilla posible (línea de comandos o interfaz mínima).

4. Requerimientos técnicos

1. El host debe desarrollarse en C sobre Linux, con un `Makefile`.
2. El código en cada microcontrolador debe desarrollarse en C o C++. No se permite utilizar MicroPython ni CircuitPython.
3. Se permite utilizar bibliotecas externas, excepto aquellas que implementen la calendarización.
4. No se permiten soluciones alambradas.
5. Los errores de acceso a memoria (`segmentation fault` o `core dumped`) penalizan cinco puntos por ocurrencia.
6. No se permite busy waiting en el host ni `delay()` como base de la temporización en los nodos.

5. Documentación

La documentación (formato libre) debe especificar lo siguiente:

1. Experimentos: comparar RMS contra EDF con una carga que solo cumpla EDF. Así como First-Fit contra Worst-Fit con una carga que produzca asignaciones diferentes entre ambas heurísticas.
2. Documento Atributo TE: los siete puntos asociados al atributo (se debe colocar pregunta y respuesta), los cuáles son los siguientes:

a) Indicar las estrategias para el trabajo individual y en equipo de forma equitativa e inclusiva en las etapas del proyecto (planificación, ejecución y evaluación).

b) Indicar la planificación del trabajo mediante la identificación de roles, metas y reglas.

c) Indicar cuáles acciones promueven la colaboración entre los miembros del equipo durante el desarrollo del proyecto.

d) Indicar cómo se ejecutan las estrategias planificadas para el logro de los objetivos.

e) Indicar la evaluación para el desempeño del trabajo individual y en equipo.

f) Indicar la evaluación para las estrategias utilizadas de equidad e inclusión.

g) Indicar la evaluación para las acciones de colaboración entre los miembros del equipo.

6. Entregables

- Código fuente con documentación interna.
- Documentación.
- Archivos de configuración.
- Conjuntos de tareas de ejemplo.
- Makefile.
- Instrucciones de compilación y carga.

7. Evaluación

Rubro	**Porcentaje**
Calendarizador local	20 %
Planificación y admisión	15 %
Comunicación y sincronización	15 %
Tareas esporádicas	5 %
Caída de nodos	10 %
Reporte y experimentos	10 %
Hardware e integración	10 %
Documentación	15 %

8. Otros aspectos administrativos

- Debe entregarse tanto la documentación como la implementación.
- No se reciben entregas tardías.
- Durante la revisión podrán utilizarse conjuntos de tareas del profesor y provocarse fallas de nodos.
- Cualquier integrante puede ser consultado sobre cualquier parte del proyecto.
- Es responsabilidad del grupo facilitar la verificación del funcionamiento.
- Las dudas sobre la especificación pueden aclararse mediante cita con el profesor.
- Los requerimientos especiales de hardware o software deben comunicarse con anticipación.
- Todos los miembros deben estar presentes en la defensa.

Si tiene dudas puede escribir al profesor al correo electrónico. **Los documentos serán sometidos a control de plagios**. La entrega se debe realizar por medio del TEC-Digital en la pestaña de evaluación. No se aceptan entregas extemporáneas después de la fecha de entrega a las 23:59.